

Conflict Management in the Near-RT-RIC of Open RAN: A Game Theoretic Approach

Abdul Wadud^{1,3}, Fatemeh Golpayegani¹, and Nima Afraz^{1,2}

¹School of Computer Science, University College Dublin, Ireland

²UCD Beijing-Dublin International College (BDIC)

³Bangladesh Institute of Governance and Management

abdul.wadud@ucdconnect.ie and *{fatemeh.golpayegani, nima.afraz}@ucd.ie*

Abstract—Open Radio Access Network (RAN) was introduced recently to incorporate intelligence and openness into the upcoming generation of RAN. Open RAN offers standardized interfaces and the capacity to accommodate network applications from external vendors through Extended Applications (xApps), which enhance network management flexibility. The Near Real Time RAN Intelligent Controller (Near-RT-RIC) employs specialized and intelligent xApps for achieving time-critical optimization objectives, but conflicts may arise due to different vendors' xApps modifying the same parameters or indirectly affecting each others' performance. A standardized Conflict Mitigation System (CMS) is absent in most of the popular Open RAN architectures including the most prominent O-RAN Alliance architecture. To address this, we propose a CMS with independent controllers for conflict detection and mitigation between xApps in the Near-RT-RIC. We utilize cooperative bargain game theory, including Nash's Social Welfare Function (NSWF) and the Eisenberg-Galle (EG) solution, to find optimal configurations for conflicting parameters. Experimental results demonstrate the effectiveness of the proposed Conflict Mitigation Controller (CMC) in balancing conflicting parameters and mitigating adverse impacts in the Near-RT-RIC on a theoretical example scenario.

Index Terms—Open RAN, Near-RT-RIC, conflict mitigation, game theory

I. INTRODUCTION

OPEN RANs are one of the most prominent architectures for 5G and beyond communication networks. They have been widely studied in recent years by experts from both academia and industry [1], [2]. The concept of Open RAN has emerged with the advent of the 5G and Internet of Things (IoT) based technologies like self-driving cars, high-speed virtual reality-based gaming, and smart factories; this is because the capabilities of the existing RAN become inadequate to fulfil the future Fifth Generation (5G)/Sixth Generation (6G) networks and IoT requirements [1], including network flexibility, ultra-low latency and massively diverse connectivity. The main motivation behind introducing Open RANs is the limitation of existing RANs due to the monolithic nature of their components, which makes them a black box to the network operators [2]. This black box provokes various disadvantages, including limited reconfigurability, restricted coordination among network nodes, and vendor lock-in at the operators' end.

These challenges lead to the inefficient utilization of spectrum resources and cause difficulties in radio spectrum management and optimization using real-time adaptation. Open RAN can be used to overcome these challenges, and it can provide network control to the operators using a centralized Software Defined Radio Access Network (SD-RAN). Also, Open RAN promotes a disaggregated and virtualized architecture, connected through open and standardized interfaces and interoperable with different vendors. Promising open, resilient, reconfigurable, cost and energy-efficient RANs that are amenable to data-driven closed-loop control using software-defined protocol stacks. Besides, Open RAN extends the opportunity of using Artificial Intelligence (AI)/Machine Learning (ML) to automate network management and maintenance.

The RAN Intelligent Controller (RIC) maintains the network control operations in open RAN, enabling various vendors to install control applications that manage specific network targets (e.g., resource allocation, energy saving, mobility load balancing, and more [2]). Non-time critical tasks that can be completed in more than 1s are performed by Remote Applications (rApps) installed in the Non Real Time RAN Intelligent Controller (Non-RT-RIC), whereas time-critical tasks that should be completed within 10ms to $\leq 1s$ are performed by xApps installed in the Near-RT-RIC. These xApps and rApps control the network operation and management within the RIC. For instance, an rApp can improve network performance and reduce latency during a Vehicle to Everything (V2X) dynamic handover scenario by adjusting radio resource allocation policies [3]. Similarly, an xApp can maximize Quality of Service (QoS) for a block of users by managing radio resources and sending dedicated control messages to the RAN nodes [4]. As these xApps and rApps can be provided by various vendors and they share the same resources concurrently during network operation, they may influence each other's performance negatively [5]. These interferences in control decisions, known as conflicts, must be detected and mitigated; otherwise, they can result in significant performance degradation within the system.

In this study, we discuss the taxonomy of possible conflicts within the Open RAN architecture. We propose a CMS that

can detect various types of conflicts in the Near-RT-RIC among xApps and mitigate them based on iterative game theoretic approaches and Nash equilibrium. The proposed CMS has an independent Conflict Detection Controller (CDC) that detects conflicts using the similar architecture proposed in [7], but it functions entirely uniquely (see section V-B). The proposed CMS in this paper uses a QoS threshold to trigger the CDC that ensures guaranteed minimum service requirement in Open RAN. The CMC component of the proposed CMS uses NSWF or EG solution to maximize the collective performance of the network. We use an experimental model in section VI to analyze the performance of the proposed CMC and discuss the functionalities of the proposed solution. Although we only deal with conflicts between xApps in the Near-RT-RIC in this article, our proposed solution can be extended to mitigate all types of conflicts in Open RAN with proper infrastructure and planning.

The remainder of this paper is structured as follows. In section II, we briefly discuss the architecture of Open RAN with all the significant components, splits, and interfaces. Section III presents a taxonomy of all possible conflicts in Open RAN with proper definitions and instances. The required infrastructure for the proposed CMS is discussed in section IV. In section V, we discuss the proposed CMS in detail. A theoretical example model is used to evaluate the performance of the proposed CMS; numerical results and analysis in this regard are discussed in section VI. Challenges and limitations of the proposed CMS are discussed in section VII. Finally, we conclude the paper in section VIII with concluding remarks and future research direction.

II. OPEN RAN ARCHITECTURE

The traditional RAN base station is segregated into three major units (see in Fig. 1), namely- open centralized unit (O-CU), open distributed unit (O-DU), and open radio unit (O-RU), which is also known as remote radio units (RRU). The O-CU is further divided into control planes and user planes. The CUs handle higher-layer protocols and larger time-scale functions. In contrast, the DUs control lower-layer protocols and time-critical tasks, and the RRU/RUs deal with physical layers and manage Radio Frequency (RF) communications and RF components. The open interface between the RU and DU is called the lower-layer splits (LLS), whereas the interface between DU and CU is called the higher-layer splits (HLS).

In the HLS, the F1 interface is used to connect the DUs with the CUs with several other dedicated sub-interfaces for user and control planes. In the LLS, the front haul control-user-synchronization (CUS) and management plane interfaces are used to connect RUs with DUs. Each interface is defined with a combination of well-organized procedures and plays a vital role in the Open RAN ecosystem by enabling different types of control and automation to the network. All required applications for these interfaces run on the RIC. The RIC is one of the significant elements for enabling various key 5G

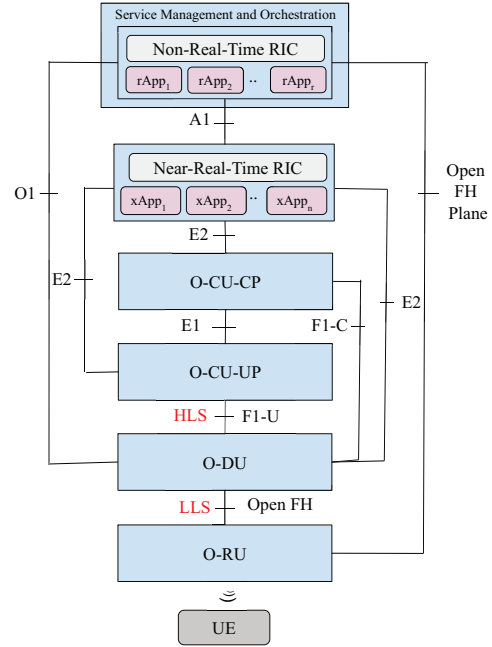


Fig. 1: Architecture of Open RAN

features, including network slicing, low-latency applications, prioritized communications, high bandwidth, etc.

In the architecture Open RAN, the RIC is categorized as Non-RT-RIC and Near-RT-RIC. The former deals with events and resources with a response time of 1 second or higher, whereas the latter manages events and resources with a response time on a scale of 10 milliseconds to 1 second. The Non-RT-RIC operates on the operators' network centrally, whereas the Near-RT-RIC can operate both centrally or on the edge of the network. A set of standardized interfaces, including A1, E1, E2, F1, O1, Open RAN front haul, and others, are used in different parts of the Open RAN architecture and enable the Non-RT-RIC and Near-RT-RIC to support the control loops [2]. As a whole, the architecture of Open RAN supports its foundational principles of openness, disaggregation, virtualization, data-driven control, and open interfaces.

Although the proposed Open RAN architecture in Fig. 1 by O-RAN Alliance (an organization of Mobile Network Operators (MNOs)) [6] is capable of handling challenges of the existing RAN architecture, it brings a few new and unique challenges. The conflict between Open RAN components is one of such challenges. Various types of conflicts among Open RAN components are discussed in the following section.

III. CONFLICTS IN OPEN RAN

In traditional RAN with a single vendor, conflicts were typically resolved internally by the vendor within their equipment. Since the entire RAN infrastructure was provided by a single vendor, they had control over the design, configuration, and optimization of the network. Any conflicts or compatibility issues were handled within their closed ecosystem, resulting in a

more streamlined prevention and resolution process. However, with the introduction of Open RAN, the network architecture allows the integration of equipment and software components from multiple vendors. While this promotes interoperability and flexibility, it also introduces potential conflicts between the various components. Each vendor may have different approaches, optimizations, or parameter configurations, which can lead to clashes when integrating their solutions within the Open RAN framework and negatively influence the RAN's performance. Therefore, these conflicts should be mitigated or avoided with proper network configuration [2]. Types of various control conflicts are discussed in the following subsections.

A. Types of Conflicts in Open RAN

According to Adamczyk et al. in [7], control decision conflicts may occur in different levels of Open RAN architecture. Conflicts in Open RAN are primarily classified into horizontal and vertical conflicts (see Fig. 2).

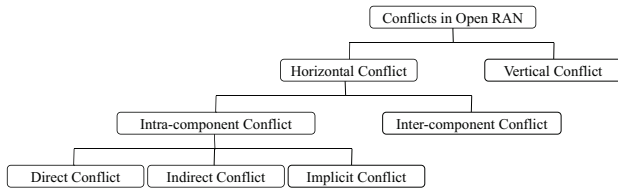


Fig. 2: Taxonomy of potential conflicts in Open RAN.

Vertical conflict occurs between components of different levels in the Open RAN architectural hierarchy. For example, the control decision conflict between a Near-RT-RIC and a Non-RT-RIC illustrated in Fig. 3 is termed a vertical conflict. Horizontal conflict, on the other hand, happens between components of the same level. For example, a conflict between two xApps in the Near-RT-RIC or between two neighboring Near-RT-RICs is considered a horizontal conflict (see Fig. 3). Control-decision conflict among xApps within a Near-RT-RIC is referred to as intra-component conflict, and conflict among xApps of neighboring Near-RT-RICs is referred to as inter-component conflict (see Fig. 3). Intra-component conflict can be further categorized into direct, indirect, and implicit conflict. In this article, we propose a mitigation technique for intra-component conflict between xApps in the Near-RT-RIC.

In the Near-RT-RIC, independent xApps functioning to achieve different optimization goals may result in conflicting configurations by modifying the same parameter or indirectly influencing the same parameter [5]; we categorize these conflicts as intra-component conflict. Detecting these direct, indirect, and implicit conflicts is difficult as the involved xApps might be developed and supplied by different vendors and are unlikely to exchange information among each other [8].

Direct conflict can be detected directly and handled by the internal conflict mitigation controller. A single xApp or two xApps may request conflicting configurations with the current running configuration. Also, two or more xApps may ask to change a particular metric to different values, which leads

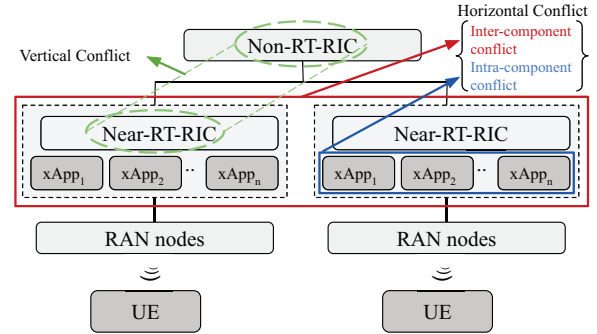


Fig. 3: Potential conflicting areas in Open RAN.

to a direct conflict. In both scenarios, the conflict mitigation controller processes these requests and decides which demand prevails in the action. This type of resolution is called the pre-action resolution [5]. However, prioritizing one request over the other is not considered an optimal solution for this problem. A more appropriate approach involves identifying an optimal configuration that addresses conflicting metrics or parameters. This not only ensures fairness but also serves the collective interest of the network to achieve its optimization goals.

On the contrary, indirect and implicit conflicts cannot be observed directly. When a change in one xApp's parameter setting affects or influences the same area of other xApps parameter changes, indirect conflict occurs. For example, cell individual offset (CIO) and antenna tilts are different control xApps, but both can affect the handover boundary. When one xApps controlling the remote electrical tilts (RET) or antenna tilts changes its values, the handover boundary in the cell is affected. This adjustment indirectly impacts the performance of the CIO controlling xApps. Mitigating this conflict requires post-action measurement by deciding an optimal value for the conflicting parameter [5].

Similarly, implicit conflict transpires when two xApps independently optimize their target but cause adversarial or unwanted effects on each other's performance. For example, two xApps maximizing QoS for a group of users and minimizing the number of handovers, respectively, may disrupt each others' performance by interfering in a non-obvious manner. This conflict is the most difficult to detect and mitigate. However, a generic approach of careful resource planning, prioritization, and error handling can help ensure the system's smooth operation [5]. In this article, we propose a conflict management system that can be used to successfully detect and mitigate all the types of conflicts, as mentioned earlier, among xApps in a Near-RT-RIC.

IV. INFRASTRUCTURE PREREQUISITES AND DATABASE

The proposed CMS demands additional components and configurations in the Open RAN architecture. Two dedicated messaging channels in the messaging infrastructure need to be configured from xApps to the Near-RT-RIC's centralized database and the conflict mitigation controller, respectively (see Fig. 4). The former redirects all control messages from

xApps to the centralized database, and the latter acts as a medium between all xApps and the CMC to find the optimal configuration of the conflicting parameter(s). Iterative communication during the optimal value calculation by the CMC with xApps and pipeline for processing all messages in the messaging infrastructure adds additional computational complexity in the Near-RT-RIC, which should be acknowledged before adopting this solution. The components of the Near-RT-RIC's database are briefly discussed in the following.

A. Database

1) *Recently Changed Parameters (RCPs)*: All recently changed parameters by the request of various xApps are stored under RCP component of the database with their respective timestamps.

2) *Parameter Group Definition (PGD)*: This component of the database stores parameters that influence the same area in the network. For example, antenna tilts and cell individual offset affect the handover boundary of a cell; therefore, both Input Control Parameters (ICPs) are included in the handover boundary group.

3) *Recently Changed Parameter Group (RCPG)*: If any parameter is changed that belongs to PGD, it is stored under RCPG component of the database with its respective timestamp and other involved parameters of the same group.

4) *Parameter and KPI Ranges (PKR)*: This component of the database stores the minimum and maximum possible value of the individual parameter and the Key Performance Indicator (KPI) of the particular cell.

5) *Decision Correlated with KPI Degradation (DCKD)*: This component of the database stores individual KPI threshold based on the QoS/Service Level Agreement (SLA) requirement of respective cells/networks.

6) *KPI Degradation Occurrences (KDO)*: This component of the database stores any KPI degradation occurrence after every parameter changes by the xApps through RAN nodes with respective timestamps.

V. CONFLICT MANAGEMENT SYSTEM

In this study, we propose a conflict management system that adopts the cooperative game theory and iterative bargaining concept to find optimal configurations for the conflicting parameters. The CMS consists of two major components, viz- the CDC and the CMC. The CDC detects conflicts with the help of a centralized database that records every action taken by an xApp with respective timestamps. The Performance Monitoring (PMon) component informs the CDC to trigger when a specific KPI is unable to meet the QoS threshold. The CDC accesses the database through a Shared Data Layer (SDL) and fetches the most recently changed parameter(s) that are responsible for the recent KPI degradation.

Two states of those parameters' values are passed to the CMC for processing an optimal configuration of the conflicting parameter(s). One state fetches the last record when the KPI

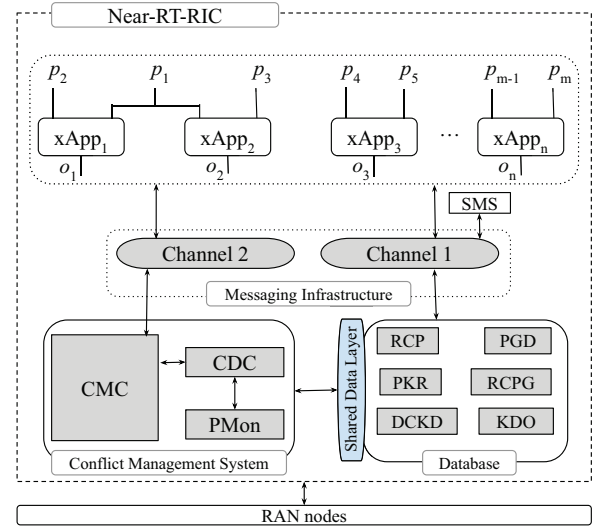


Fig. 4: Architecture of CMS in the Near-RT-RIC.

was above the QoS threshold, and the other fetches the most recent record after the degradation transpired. The CDC detects the exact parameter responsible for that specific KPI degradation using the PGD table and RCP table and passes the above-stated two records to the CMC. Afterwards, the CMC communicates respective xApp(s) dealing with that specific parameter through a dedicated messaging channel and suggests an optimal configuration for the conflicting parameter using a bargain game theoretic approach. The following subsections discuss the functionalities of the proposed CMS with appropriate figures and examples.

A. Performance Monitoring Component (PMon)

The PMon component in the CMS is responsible for monitoring Key Performance Indicators (KPIs) to assess and analyze the performance of the network. The component collects data from various RAN nodes (see in Fig. 4) that include network elements, user devices, and other monitoring probes. This data includes measurements, statistics, and events related to network performance. The collected data is processed and analyzed to calculate the values of the defined KPIs. This involves applying algorithms and calculations to the collected data to derive meaningful performance metrics. Afterwards, the performance monitoring component compares the calculated KPI values against predefined QoS thresholds or target values. QoS thresholds define acceptable ranges for each KPI, and if a KPI falls outside the defined QoS range, it indicates a performance deviation or anomaly. These performance deviation occurrences are recorded in the KDO component of the database. The PMon component sends a trigger to the CDC when certain KPIs breaches predefined QoS thresholds. Afterwards, the CDC starts looking for possible conflicts between xApps and their respective parameters.

B. Conflict Detection Controller (CDC)

The presented CDC in Fig. 4 is capable of detecting direct, indirect, and implicit conflicts between xApps in the Near-RT-RIC. However, the presented CDC may detect all types of conflicts in Open RAN with proper infrastructure and planning. When deploying an xApps, direct conflict can be detected through close observation of all ICPs. However, human error may occur during the deployment, and conflicted parameter(s) can be ignored. Therefore, a detection technique is essential as part of the fail-safe mechanism to detect direct conflicts.

When a KPI degradation occurrence outside the range of QoS threshold is detected by the PMon, the CDC fetches the most recently changed parameter. Afterwards, the PGD of the database is checked for finding the affiliation of that parameter to any of the defined groups. If any affiliation is found, it is considered an indirect conflict. Otherwise, the CDC searches all recently changed parameters in the RCP to check whether this parameter was changed by any other or the same xApps in the recent past. If any match is found, it is considered a direct conflict. Otherwise, it is considered an implicit conflict. Eventually, the CDC informs the CMC about the transpired conflict with all relevant information about the conflicting parameter.

C. Conflict Mitigation Controller (CMC)

The CMC adopts cooperative game theory to deal with various types of intra-component conflicts in the Near-RT-RIC. When a direct conflict is detected between xApps, the conflicting parameter's new value is suggested by the CMC in such a way that the collective utility of all involved xApps is maximum. By utility, we refer to the KPIs of each xApp. The collective utility is calculated using the Social Welfare Function (SWF) discussed in section V-E. Similarly, an optimal value of the conflicting parameter is estimated using the SWF when an indirect or implicit conflict is detected for a recently changed parameter. The default value or the second last changed value and their respective KPIs are considered with the most recently changed value of the conflicting parameter and relevant KPI for the optimal value calculation. The most recently changed parameter refers to the parameter that is responsible for KPI degradation.

D. Theoretical Model, Assumptions and Notations

The Open RAN envisions applying AI/ML techniques to automate the network management and optimization process. To achieve this, Open RAN can leverage the techniques and capabilities of Self Organising Network (SON), which is a network management and optimization technique that aims to automate the network operation. SON deploys multi-vendor SON Functions (SFs) that are similar to xApps and rApps in Open RAN to achieve specific set of objectives.

Table I presents the list of notations used in the paper.

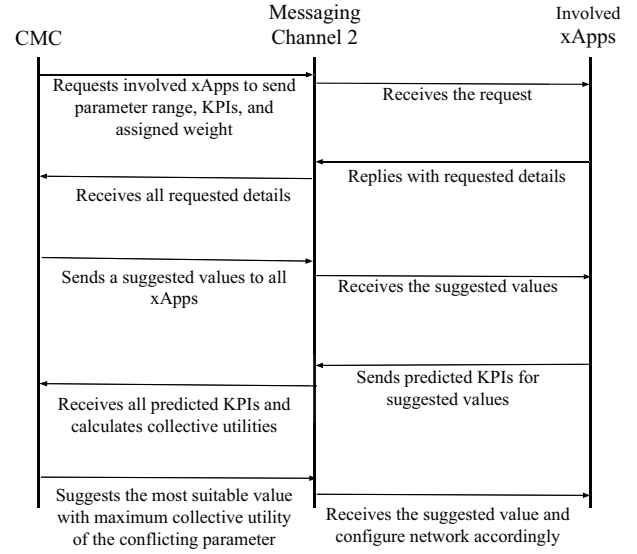


Fig. 5: Closed-loop communication between the CMC and involved xApps through the dedicated Channel 2 during optimal value calculation of the conflicting parameter.

TABLE I: List of Notations

Symbol	Description
Given Parameters:	
X	Set of all xApps in the Near-RT-RIC
P	Set of all ICPs belonging to the xApps in the Near-RT-RIC
$p \in P$	An ICP in the Near-RT-RIC
O	Set of all KPIs belonging to the xApps
$o \in O$	A KPI to measure the performance of xApps in the Near-RT-RIC
Z	Set of all conflicting xApps
J	Set of all KPIs for an individual xApp
u	Utility of an xApp converted from KPIs belonging to J
$f(x)$	Utility function to convert KPIs to utility u for a given value x of the conflicting parameter
w	Priority weight assigned by the MNO to a xApp
Decision Variables:	
x	Optimal value for the conflicting parameter

We assume each xApps similar to a SF in SON which carries four distinct properties. Firstly, each xApps is capable of learning, predicting, and deciding its best possible action by itself based on any dynamic network state s . Secondly, xApps are independent and do not communicate with each other. Thirdly, xApps are provided by multiple vendors, and there are no team or group actions by any two or more xApps. Lastly, all xApps share the same resources and have definite conflicts of interest among them. In addition, individual xApps can predict their respective KPIs based on all of their ICP's values.

We further assume that an xApps sends the following information in response to a request by the CMC through the dedicated messaging channel (channel 2 in Fig. 4). The first message from a xApps to the CMC contains the KPIs (output parameter of the xApps) for the current value of the

conflicting parameter, ranges of the conflicting parameter for a particular xApps, and a priority weight assigned by the MNO for a certain network state s . Later messages contain only the KPIs for suggested conflicting parameter's value by the CMC (see in Fig. 5). Suppose there is n number of xApps installed in a Near-RT-RIC. We represent the set of all xApps as $X = \{xApp_1, xApp_2, \dots, xApp_n\}$, set of all ICPs as $P = \{p_1, p_2, \dots, p_m\}$, and set of all output parameters (KPIs) as $O = \{o_1, o_2, \dots, o_n\}$. If there is direct conflict over a parameter p_1 in between xApps $xApp_1$ and $xApp_2$ for a certain network state s , xApps $xApp_1$ sends $[\{p_1^{min, xApp_1}, p_1^{max, xApp_1}\}, o_1^s, w_1^s]$. Similarly, $xApp_2$ sends $[\{p_1^{min, xApp_2}, p_1^{max, xApp_2}\}, o_2^s, w_2^s]$. Here, w_1^s and w_2^s are weight metrics assigned by the MNO or chosen by an automated process to prioritize one xApps over the other based on the necessity of the network state, and $w_1^s + w_2^s = 1$. We assume these weights as the bargaining power of xApps $xApp_1$ and $xApp_2$. Set of conflicting xApps is represented by Z , which is $Z = \{xApp_1, xApp_2\}$ in this case.

As each individual KPI has different units and measurement ranges, the CMC recommends converting them to a uniform scale. For instance, the KPIs can be transformed using min-max normalisation on a scale of $[0, 10]$ and defined as utility using this formula: $f_i(x) = u_i = \frac{1}{|J|} \sum_{j \in [1, |J|]} (\frac{o_{ij} - o_{ij}^{min}}{o_{ij}^{max} - o_{ij}^{min}} \times 10)$. Here, J is the set of KPIs for an individual xApps, and x is the suggested value of the conflicting parameter. For example, the performance of a power allocation xApps can be measured using Signal-to-Interference-plus-Noise Ratio (SINR), energy efficiency, and throughput; therefore, $|J| = 3$ in this case. If there is only one KPI for evaluating the performance of an xApps, $|J| = 1$. Values for o_{ij}^{min} , and o_{ij}^{max} are obtained from PKR of the database. The following section discusses how Nash's bargain game theoretic approach can be adopted for estimating the optimal value of the conflicting parameter.

E. Game Theory for Optimal Value Calculation

It is well-known that Nash's equilibrium can be computed to find the maximum overall satisfaction or collective utility among all players or agents in a multi-player game or multi-agent scenario. In Fig. 6, we consider such a conflicting scenario with two xApps for demonstration purposes only where a direct conflict over p_1 between $xApp_1$ and $xApp_2$ is detected. When the CMC received a trigger about this conflict, it communicates the involved xApps immediately and requests for their KPIs, conflicting parameters range, and weights assigned by the MNO for current network state (e.g., $o_1, \{p_1^{max, xApp_1}, p_1^{min, xApp_1}\}, w_1$ and $o_2, \{p_1^{max, xApp_2}, p_1^{min, xApp_2}\}, w_2$). The outputs are transformed into a scaled utility of similar units as discussed in section V-D.

The CMC calculates optimal range for p_1 considering both ranges of xApps as- $(p_1^{min, opt}, p_1^{max, opt}) \simeq \{min(p_1^{min, xApp_1}, p_1^{min, xApp_2}), max(p_1^{max, xApp_1}, p_1^{max, xApp_2})\}$. A sample set is prepared within this range and is used for finding the optimal value for p_1 . For each suggested value of

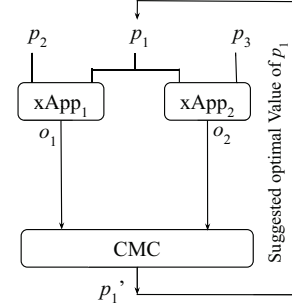


Fig. 6: Demonstration of closed-loop control for optimal value calculation by CMC.

the sample set, collective utility is calculated using an SWF. NSWF can calculate the collective utility using the product of the individual utilities of n participating xApps. The formula is represented as follows [9]:

$$NSWF(x) = \prod_{i \in [1, |Z|]} f_i(x), \forall i \in Z \quad (1)$$

In Eq. 1, NSWF intends to balance fairness and efficiency between xApps. However, it fails to reach optimal equilibrium when a particular xApps requires preference over the other conflicting xApps due to the network demand. The optimal market equilibrium, in this case, can be reached using our modified EG convex linear programming problem as follows [10], [11]:

$$\max : F = \sum_{i \in [1, |s|]} w_i f_i(x) \quad (2a)$$

$$\text{s.t.} \quad \sum_{i \in [1, |Z|]} w_i = 1, \forall i \in Z \quad (2b)$$

$$x \geq p_m^{min, opt}, m \in P \quad (2c)$$

$$x \leq p_m^{max, opt}, m \in P \quad (2d)$$

$$\forall i \in [1, |Z|], |Z| \geq 2. \quad (2e)$$

Here, F represents the objective function, and x indicates the decision variable for obtaining the optimal configuration of the conflicting parameter. The value of x is constrained within the optimal range of conflicting parameters. The following section analyses the performance of both approaches to solving various conflicts.

F. Use cases of NSWF and EG

Let us consider a scenario where four xApps for Mobility Load Balancing (MLB), Capacity and Coverage optimization (CCO), Energy Saving (ES), and Mobility Robustness optimization (MRO) are functioning in a Near-RT-RIC of Open RAN. These xApps aim to achieve their specific KPI objective of reducing network congestion, optimizing capacity and coverage, obtaining energy efficiency, and reducing packet/call drop rates or the number of unsuccessful handovers, respectively. Transmission power (TxP) is a common ICP or resource that all of these xApps share while functioning. The NSWF solution can help to suggest a value for the TxP that ensures maximum collective utility in the network.

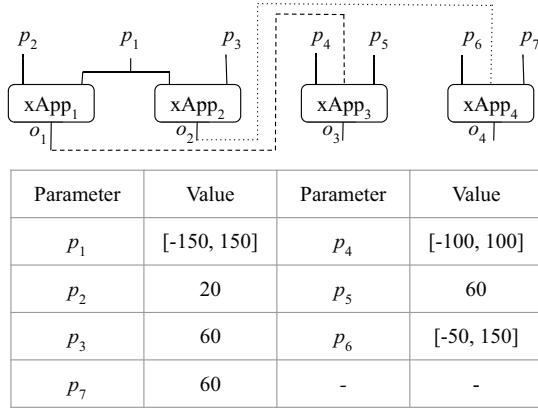


Fig. 7: Example model for direct $\{xApp_1$ and $xApp_2\}$, indirect $\{xApp_2$ and $xApp_4\}$ and implicit $\{xApp_1$ and $xApp_3\}$ conflict.

This collective utility is obtained through a trade-off between different KPIs, such as reducing energy efficiency or suffering network congestion for a certain period to reduce the number of unsuccessful handovers or call drop rates.

Suppose the MNO is experiencing a high level of call drop rate in a particular cell or in a handover boundary. The Eisenberg-Galle solution can be adopted by the MNO to prioritize the MRO xApps over the other xApps. An increment in the transmission power (TxP) will lead to higher throughput in that particular cell, or an increment in the cell's individual offset value to broaden the handover boundary can minimize the number of call drops.

VI. NUMERICAL RESULTS AND ANALYSIS

To assess the performance of the proposed CMC, we construct an example model incorporating direct, indirect, and implicit conflicts, as illustrated in Fig. 7. During the numerical analysis, we assume that the output of all xApps, representing the KPIs, follows a Gaussian distribution function, as it often mirrors real-life scenarios [10]. The equations ($o_1 = 0.5 \times e^{-\frac{(p_1+50)^2}{2p_2^2}}$, $o_2 = e^{-\frac{(p_1-50)^2}{2p_3^2}}$, $o_3 = e^{-\frac{(p_4+o_1)^2}{2p_5^2}}$, $o_4 = e^{-\frac{(p_7+o_2)^2}{2p_6^2}}$, $o'_1 = 0.6 \times e^{-\frac{(p_1-85)^2}{2p_2^2}}$, $o'_2 = 0.35 \times e^{-\frac{(p_1+0)^2}{2p_3^2}}$) and parameters are configured to accurately represent the mentioned conflicts. Indirect and implicit conflicts are modeled similarly, as it is challenging to precisely model implicit conflict, and both types can be mitigated using a similar approach.

All obtained KPIs from the xApps in Fig. 7 are converted to utilities within the range $[0, 1]$ using the formula stated in section V-D. Priority weights assigned by the MNO to xApps for direct, indirect, and implicit conflicts are assumed to be $\{xApp_1 = 0.4$ and $xApp_2 = 0.6\}$, $\{xApp_2 = 0.1$ and $xApp_4 = 0.9\}$, and $\{xApp_1 = 0.9$ and $xApp_3 = 0.1\}$, respectively. These weights are utilized only for the EG solution when the MNO prioritizes an xApp over others based on the network state and demands. The NSWF and EG solutions are effective in non-priority and priority scenarios, respectively. We perform

Arithmetic Mean (AM) with different values for the conflicting parameter and obtain the optimal value for comparison with the performance of NSWF in a non-priority scenario. Each intelligent xApps is trained using Polynomial Regression Blocks in Python, leveraging the generated dataset available in [12] through the aforementioned equations.

In Fig. 8(a), we illustrated the direct conflict mitigation between $xApp_1$ and $xApp_2$ over the parameter p_1 . Based on the current network state assumed in the example model, $xApp_1$ demands $p_1 = 50$ to obtain its objective, whereas $xApp_2$ demands $p_1 = -50$; this creates a direct conflict between these two xApps. To resolve this conflict, we used NSWF, AM, and EG solutions to suggest a value for p_1 as demonstrated in Fig. 8(a). The NSWF ensures maximum collective utility over AM and EG, whereas EG prioritizes $xApp_2$'s utility over $xApp_1$'s as per the priority weight assigned by the MNO.

To demonstrate the capability of the proposed CMS in handling indirect conflicts, we focus on $xApp_2$ and $xApp_4$. The utility functions of these xApps are designed to have a logical dependency, leading to an indirect conflict scenario. When $xApp_2$ modifies parameter p_1 at timestamp t_2 from its previous value at t_1 , it indirectly impacts the scope of $xApp_4$, resulting in a reduction of its utility as illustrated in Fig. 8(b). The change made by $xApp_2$ at t_2 , setting $p_1 = 0$ based on the network state, indirectly affects the utility of $xApp_4$, nearly reducing it to zero.

This resembles the impact of the energy-saving xApp altering transmission power and indirectly influencing the handover boundary, thereby reducing the utilities associated with mobility robustness optimization xApp. Comparing the optimal values suggested by AM and NSWF, we observe that AM does not improve the utility of $xApp_4$, while NSWF significantly enhances the collective utility between the two conflicting xApps (see Fig. 8(b)). Additionally, the EG solution verifies higher utility for $xApp_4$ when it is prioritized by the MNO.

To analyze the implicit conflict between $xApp_1$ and $xApp_3$, we formulate the utility equation for $xApp_3$ to capture any inadvertent or unwanted impact, similar to an indirect conflict resulting from $xApp_1$ changing p_1 . As depicted in Fig. 8(c), the implicit conflict arises when $xApp_1$ sets $p_1 = -85$ at timestamp t'_2 from its value at t'_1 , leading to a reduction in the utility of $xApp_3$ below 0.4, causing dissatisfaction and triggering the CMC. Subsequently, the CMC suggests optimal values for p_1 using AM, NSWF, and EG. Both AM and NSWF improve the utility for $xApp_3$ while reducing that of $xApp_1$. On the other hand, the EG solution enhances the utility for $xApp_1$ when it is prioritized by the Mobile Network Operator (MNO), establishing a balance between the utilities of $xApp_1$ and $xApp_3$ at t'_2 .

The indirect conflict occurs when $xApp_2$ requests a different setting of p_1 at timestamp t_2 while the $xApp_4$ remains with the same setting. Therefore, we only illustrated changes in $xApp_2$'s utilities at different timestamps in Fig. 8(b). Similar explanation is applied to $xApp_1$ and $xApp_3$ in Fig. 8(c).

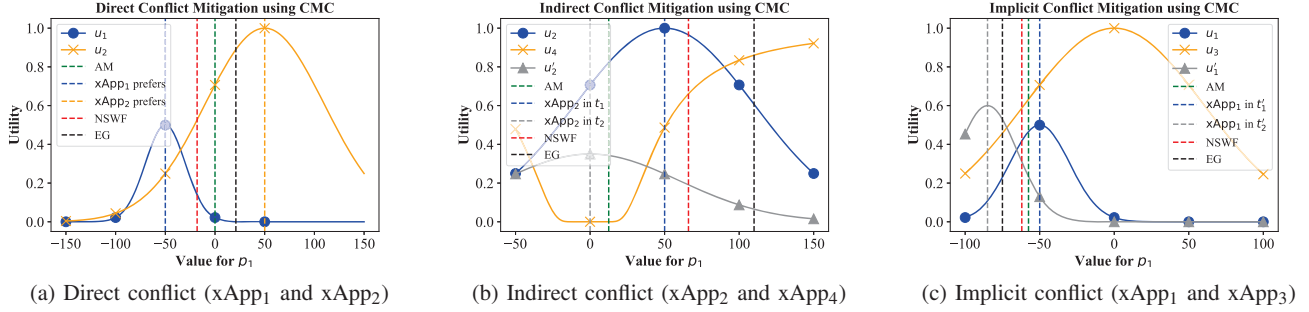


Fig. 8: Conflict mitigation using the proposed CMC.

VII. LIMITATIONS AND CHALLENGES

After critically analyzing the current state of the proposed conflict mitigation system, we can underline the following limitations and challenges of the proposed approach:

- A Higher number of xApps and ICPs involved in a conflicting scenario may increase complexity to reach the optimal equilibrium state and hence may exceed the time threshold of Near-RT-RIC functions.
- Converting KPIs to utilities can be a challenge in a real-world setting.
- The proposed solution is only tested in a theory-based example scenario which does not assure its applicability in a real-world system.
- The deployment of the proposed solution has a specific infrastructure prerequisite to be fulfilled.
- The benefit of the proposed approach can only become conclusive after its successful deployment in a real-world setting with real xApps.

VIII. CONCLUSION AND FUTURE WORKS

In this article, we have presented a novel game-theory based CMS for the Near-RT-RIC to ensure an intelligent and vendor-agnostic Open RAN architecture. Our proposed system is designed to address the issue of conflicts arising from the coexistence of multiple xApps from different vendors. The core contribution of our work lies in the establishment of an independent conflict detection and mitigation controller to mitigate conflicts. By introducing a game-theory-based independent conflict mitigation controller, we demonstrate effective mitigation of conflicts between xApps in an experimental setup.

The application of cooperative bargain game theory, specifically the NSWF and the EG solution, allowed us to determine optimal configurations for conflicting parameters, ensuring maximum collective utility for the network. Moving forward, we intend to transition from the theoretical model to a more practical real-world setup and overcome the current limitations of the proposed approach. We plan to reduce the complexity and computational overhead for scenarios with a high density of xApps and ICPs. Additionally, we plan to explore machine

learning techniques to predict probable conflicts and multi-agent reinforcement learning techniques for mitigating the detected conflicts.

ACKNOWLEDGMENT

We acknowledge funding from the European Union's Horizon Europe research and innovation program under the Marie Skłodowska-Curie SE grant agreement RE-ROUTE No 101086343.

REFERENCES

- [1] Singh, Sameer Kumar, Rohit Singh, and Brijesh Kumbhani. "The evolution of radio access network towards open-RAN: Challenges and opportunities.", (WCNCW), pp. 1-6. IEEE, 2020.
- [2] Polese, Michele, Leonardo Bonati, Salvatore D'Oro, Stefano Basagni, and Tommaso Melodia. "Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges." IEEE Communications Surveys & Tutorials (2023).
- [3] Dryjański, Marcin. "Traffic Management for V2X Use Cases in O-RAN." Available online: <https://rimedolabs.com/blog/traffic-management-for-v2x-use-cases-in-o-ran/> [Accessed on: 09-11-2023].
- [4] O-RAN Alliance, O-RAN Use Cases Analysis Report, O-RAN Alliance, October 2022, v09.00.
- [5] O-RAN Working Group 3, Near-Real-time RAN Intelligent Controller Architecture, O-RAN Alliance, October 2022, v03.00.
- [6] O-RAN Working Group 1, O-RAN Architecture Description, O-RAN Alliance, October 2022, v07.00.
- [7] Adamczyk, Cezary, and Adrian Kliks. "Conflict Mitigation Framework and Conflict Detection in O-RAN Near-RT RIC." IEEE Communications Magazine (2023).
- [8] Zhang, Han, Hao Zhou, and Melike Erol-Kantarci. "Team learning-based resource allocation for open radio access network (O-RAN)." In ICC 2022-IEEE International Conference on Communications, pp. 4938-4943. IEEE, 2022.
- [9] Ramezani, Sara, and Ulle Endriss. "Nash social welfare in multiagent resource allocation." In International Workshop on Agent-Mediated Electronic Commerce, pp. 117-131. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009.
- [10] Banerjee, Anubhab, Stephen S. Mwanje, and Georg Carle. "Toward control and coordination in cognitive autonomous networks." IEEE Transactions on Network and Service Management 19, no. 1 (2021): 49-60.
- [11] Brânzei, Simina, Vasilis Gkatzelis, and Ruta Mehta. "Nash social welfare approximation for strategic agents." In Proceedings of the 2017 ACM Conference on Economics and Computation, pp. 611-628. 2017.
- [12] <https://github.com/dewanwadud1/cmsORAN>.